

Lập trình hướng đối tượng (OOP) trong Java

- Là phương pháp thiết kế chương trình dựa trên các Lớp (Class) và Đối tượng (Object),
- tập trung vào việc tổ chức mã nguồn thành các thực thể thực tế với thuộc tính (dữ liệu) và phương thức (hành động), giúp code dễ bảo trì, tái sử dụng và mở rộng nhờ 4 tính chất cốt lõi (Pillars of OOP):
 1. *Đóng gói (Encapsulation),*
 2. *Kế thừa (Inheritance),*
 3. *Đa hình (Polymorphism), và*
 4. *Trừu tượng (Abstraction).*

Các khái niệm cơ bản

- **Đối tượng (Object):** Một thể hiện (instance) của một lớp, có trạng thái (thuộc tính) và hành vi (phương thức) riêng.
 - Ví dụ: một đối tượng SinhVien có tên, mã số (thuộc tính) và có thể thi (phương thức).
- **Lớp (Class):** Một bản thiết kế, khuôn mẫu để tạo ra các đối tượng. Định nghĩa các thuộc tính và phương thức chung cho các đối tượng thuộc lớp đó.

Tính đóng gói (Encapsulation)

- Dữ liệu **không** cho truy cập trực tiếp
- Chỉ thao tác thông qua **getter / setter**
- Kiểm soát tính hợp lệ của dữ liệu

```
Public Student (String name, int age){  
    this.name = name;  
    this.age = age;  
}
```

student.age = -5

```
public class Student {  
    // Thuộc tính để private → không cho truy cập trực tiếp  
    private String name;  
    private int age;  
  
    // Constructor  
    public Student(String name, int age) {  
        this.name = name;  
        setAge(age); // kiểm soát dữ liệu qua setter  
    }  
  
    // Getter  
    public String getName() {  
        return name;  
    }  
  
    // Setter có kiểm tra điều kiện  
    public void setAge(int age) {  
        if (age > 0) {  
            this.age = age;  
        } else {  
            System.out.println("Tuổi không hợp lệ!");  
        }  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

Tính kế thừa (Inheritance)

- Lớp con **kế thừa** lại code của lớp cha
- Tránh viết lại code giống nhau . tái sử dụng code

```
public class Main {  
    public static void main(String[] args) {  
        // Tạo đối tượng Dog  
        Dog dog = new Dog("Lucky");  
  
        // Gọi phương thức kế thừa từ Animal  
        dog.eat();  
  
        // Gọi phương thức riêng của Dog  
        dog.bark();  
    }  
}
```

```
// Lớp cha  
public class Animal {  
    protected String name;  
  
    public void eat() {  
        System.out.println("Động vật đang ăn...");  
    }  
}
```

```
// Lớp con kế thừa Animal  
public class Dog extends Animal {  
  
    public Dog(String name) {  
        this.name = name;  
    }  
  
    public void bark() {  
        System.out.println(name + " đang sủa: Gâu gâu!");  
    }  
}
```

Đa hình (Polymorphism)

- Cùng 1 lời gọi phương thức
- Nhưng hành vi khác nhau
- Xảy ra khi override phương thức

```
// Lớp cha
public class Animal {
    public void makeSound() {
        System.out.println("Động vật phát ra âm thanh...");
    }
}
```

```
// Lớp con Dog
public class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Gâu gâu!");
    }
}
```

```
// Lớp con Cat
public class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Meo meo!");
    }
}
```

Dog myDog = new Dog();
Cat myCat = new Cat();

```
public class Main {
    public static void main(String[] args) {

        Animal a1 = new Dog();
        Animal a2 = new Cat();

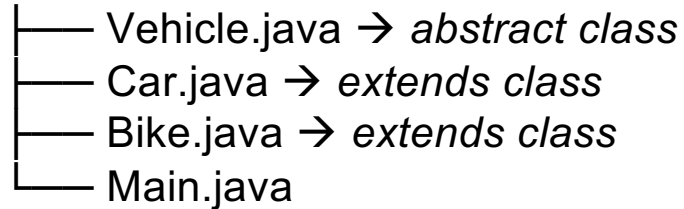
        a1.makeSound(); // Gâu gâu
        a2.makeSound(); // Meo meo
    }
}
```

Trừu tượng (Abstraction)

- Không quan tâm làm thế nào
- Chỉ quan tâm *làm cái gì* ✓
- Dùng abstract class hoặc interface

- Cách 1: Abstract class

vehicle-demo/



```
public class Car extends Vehicle {  
    @Override  
    public void move() {  
        System.out.println("Xe hơi chạy bằng 4 bánh");  
    }  
}
```

```
public abstract class Vehicle {  
  
    // Phương thức trừu tượng  
    public abstract void move();  
  
    // Phương thức thường  
    public void start() {  
        System.out.println("Phương tiện bắt đầu chạy");  
    }  
}
```

```
public class Bike extends Vehicle {  
    @Override  
    public void move() {  
        System.out.println("Xe máy chạy bằng 2 bánh");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        Vehicle v1 = new Car();  
        Vehicle v2 = new Bike();  
  
        v1.move();  
        v2.move();  
    }  
}
```

// abstract class không thể tạo đối tượng trực tiếp → phải tạo đối tượng của lớp con

- Cách 2: Interface

payment-demo/

- Payment.java → *interface*
- CashPayment.java → *class implements*
- OnlinePayment.java → *class implements*
- CardPayment.java → *class implements*
- Main.java

Khi nào nên dùng **abstract class** và khi nào nên dùng **interface**?

👉 Dùng **abstract class** khi:

1. Các lớp con có mối quan hệ **"is-a"** chặt chẽ
(ví dụ: `Dog` là một `Animal`, `Car` là một `Vehicle`)
2. Cần **chia sẻ code** cho các lớp con
 - Có thuộc tính chung
 - Có phương thức đã cài đặt sẵn
3. Muốn **kết hợp giữa trừu tượng và cụ thể**
 - Có phương thức **abstract**
 - Có phương thức bình thường
4. Không cần kế thừa từ nhiều lớp khác
(vì Java chỉ cho kế thừa **1 class**)

➡ Phù hợp khi **mọi Vehicle** đều có **speed, start(), move()**

Khi nào nên dùng **abstract class** và khi nào nên dùng **interface**?

👉 Dùng interface khi:

1. Chỉ muốn định nghĩa **hành vi (what)**, không quan tâm cài đặt (how)
2. Cần đa kế thừa
 - Một lớp có thể `implements` **nhiều interface**
3. Các lớp **không liên quan về mặt phân cấp**, nhưng có cùng hành vi (ví dụ: `Car`, `Robot`, `Drone` đều có thể `Moveable`)
4. Thiết kế theo hướng **mở rộng, linh hoạt, plugin**

➡ `CashPayment`, `OnlinePayment`, `CryptoPayment` đều có thể triển khai `pay()`